

Web Scraping, Web Crawling & APIs

Pourquoi la collecte de données importe en IA/Data : modèles, features, évaluation et surveillance dépendent de données pertinentes et reproductibles.

Trois voies principales pour récupérer les données du Web

Web Scraping → extraire des pages • Web Crawling → explorer et découvrir • API → interroger un service structuré

- 📌 Notes orales : Poser le contexte — l'IA dépend d'un flux de données propre et légal. Présenter ensuite les trois approches et quand les préférer.

Web Scraping — Définition et fonctionnement

Définition : Le Web Scraping est une technique qui extrait automatiquement des informations spécifiques présentes dans le code HTML d'une page.

01

Étapes (flux)

Requête HTTP → réception du HTML
→ parsing → sélection d'éléments →
extraction → nettoyage → stockage

03

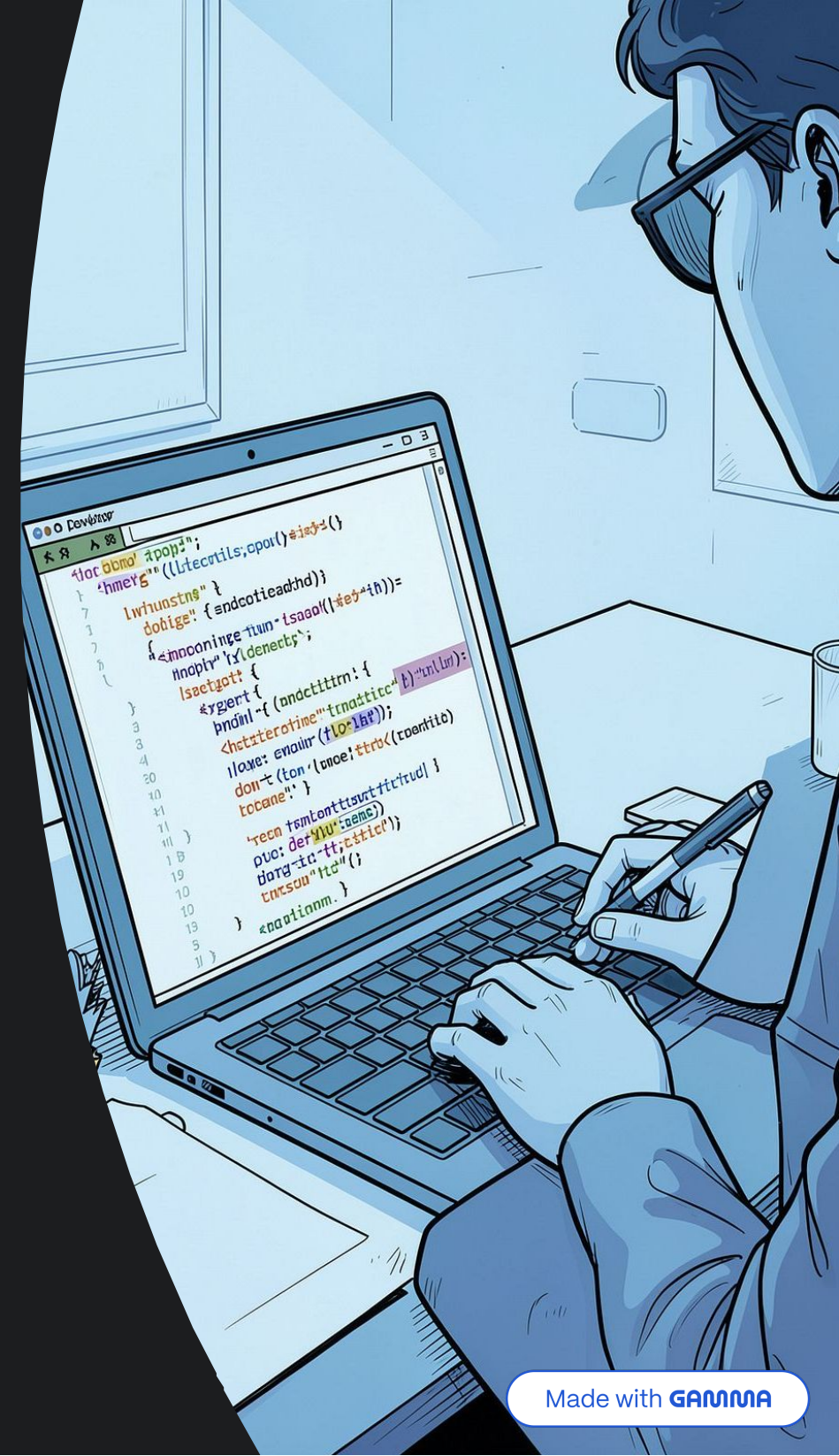
Pages statiques vs dynamiques

Statique : contenu dans le HTML. Dynamique : contenu rendu/modifié par JavaScript (requiert navigateur automatisé).

02

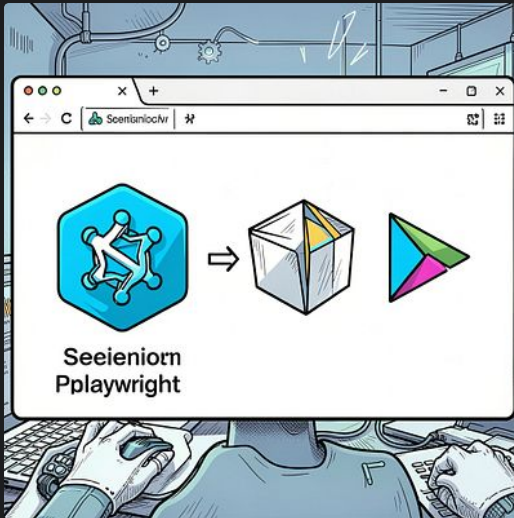
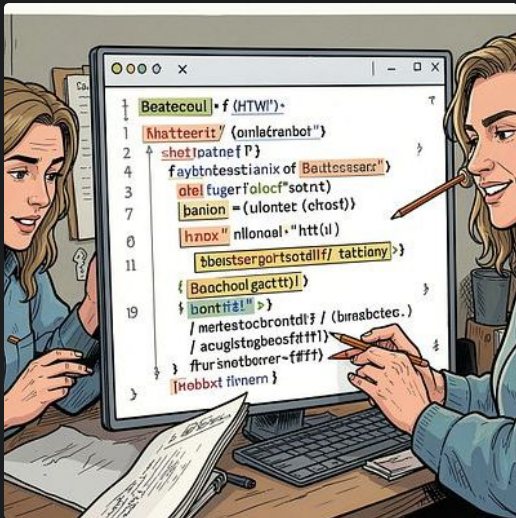
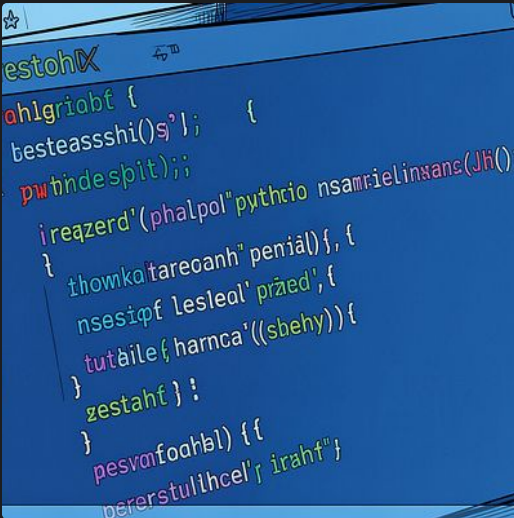
Exemple concret

E-commerce : Nom produit | Prix |
Note | Disponibilité — le scraper
récupère ces champs ciblés.



- Notes orales : Insister sur la chaîne technique et sur la différence pratique — un simple Requests+BS4 suffit pour une page statique, sinon Selenium/Playwright.

Bibliothèques & outils pour le Web Scraping



Outil	Usage clé (phrases courtes)
Requests	Client HTTP — envoyer GET/POST et récupérer HTML.
BeautifulSoup	Parsing HTML/XML — sélection par balises, classes, ids, sélecteurs CSS.
lxml	Parser performant, XPath supporté — utile pour extraction précise.
Scrapy	Framework complet (spiders, pipelines, middlewares) pour scraping à grande échelle.
Selenium	Automatisation de navigateur — simulation d'interactions et rendu JS.
Playwright	Automatisation moderne multi-browser, fiable pour applications dynamiques.
Plateformes	Apify, ParseHub, Octoparse — solutions SaaS low/no-code pour scrapes rapides.

Rappel : Requests = HTTP • BeautifulSoup = parsing • Scrapy = framework • Selenium/Playwright = navigateur • Apify/ParseHub = plateformes

Notes orales : Donner exemples d'usage — petit projet (Requests+BS4) vs pipeline industriel (Scrapy + pipelines + proxys).

Web Crawling — Définition & composants

Définition : Processus automatisé d'exploration de pages Web en suivant les liens pour découvrir et indexer des ressources.

Seed URLs

Crawler

Extract Links

URL Queue

Concepts clés

Seed URLs, URL Queue (frontier), Visited set, Link extraction, Depth, Domain restriction, Duplicate detection, Crawl delay, robots.txt

Bonnes pratiques

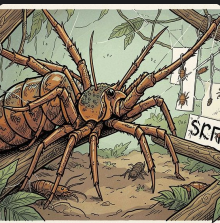
Respecter robots.txt, limiter la fréquence (crawl delay), détecter duplicatas, limiter la profondeur et surveiller l'impact serveur.

- ❏ Notes orales : Expliquer la logique de file d'attente des URLs et l'importance d'un comportement responsable (limiter taux, respecter règles légales).

Outils & frameworks de Web Crawling

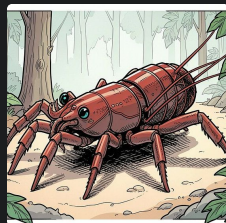


Note : Requests + BeautifulSoup peuvent être des composants d'un crawler, mais ils ne remplacent pas un framework complet.



Scrapy

Framework Python spécialisé (spiders, pipelines, middlewares) — idéal pour crawling à grande échelle.



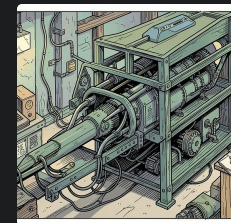
Crawlee

Framework moderne pour
construire crawlers et
scrapers, ergonomique et
extensible.



Apache Nutch

Open source, adapté à
l'indexation à grande échelle
(intégration
Hadoop/Elasticsearch).



Heritrix

Crawler pour archivage Web
— robuste pour captures
massives.



Apify

Plateforme pour exécuter et orchestrer crawlers/scrapers (SaaS).

- Notes orales : Comparer cas d'usage — Scrapy pour pipelines programmés ; Nutch pour indexation massive ; Apify pour délégation SaaS.

API — Accéder aux données autrement

Définition : Interface qui permet à deux logiciels de communiquer selon un contrat (endpoints, méthodes et formats).

1

Schéma

Application Python → requête
HTTP → API (serveur) →
réponse JSON → application

2

Concepts essentiels

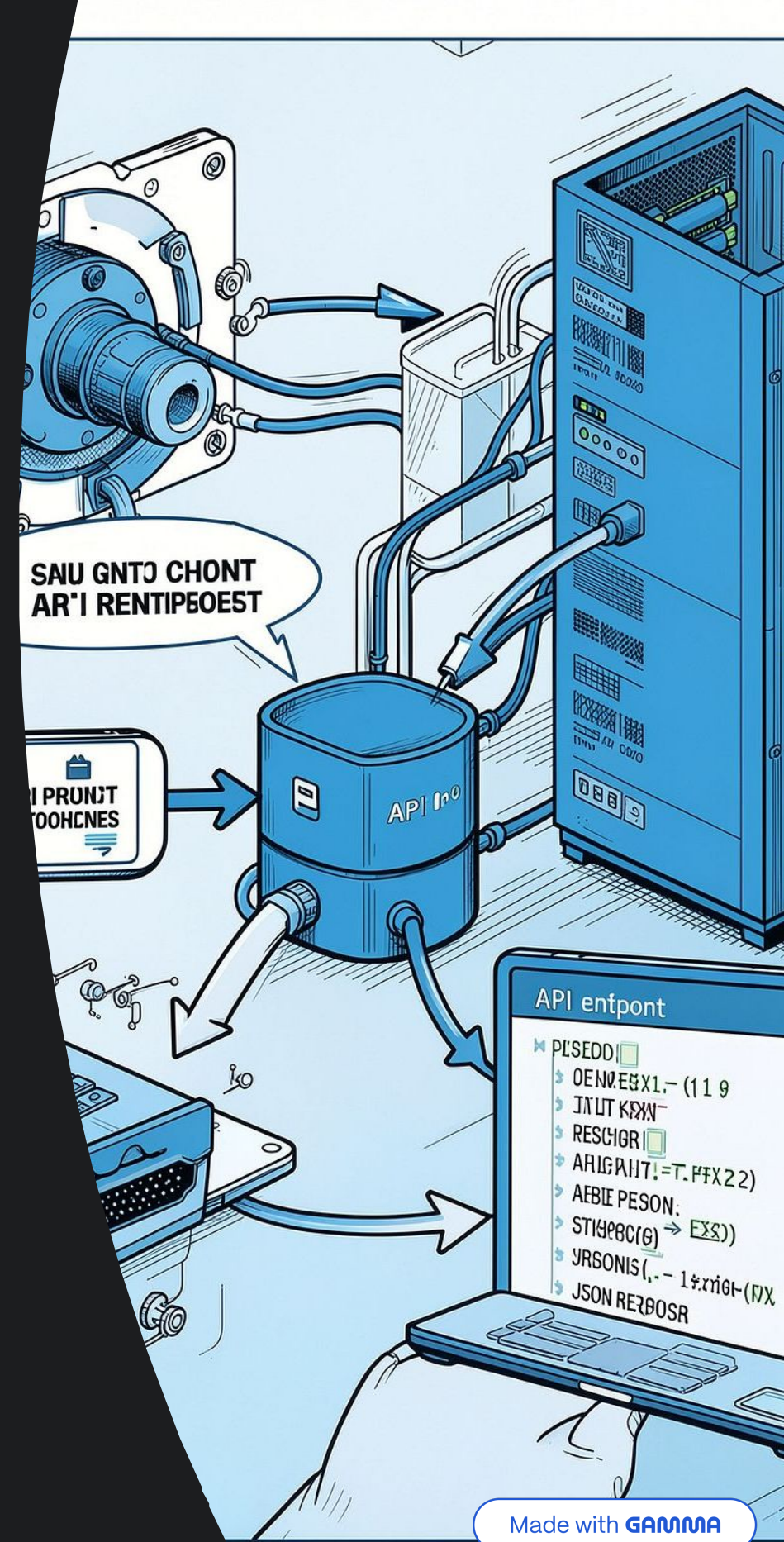
Endpoint, méthodes HTTP
(GET, POST, PUT/PATCH,
DELETE), paramètres, headers,
authentication (API Key,
OAuth), JSON, status codes.

3

Codes HTTP courants

200 OK • 201 Created • 400 Bad Request • 401 Unauthorized • 403
Forbidden • 404 Not Found • 429 Too Many Requests • 500 Internal
Server Error

- ❏ Notes orales : Insister sur la stabilité et la structure offertes par les APIs — privilégier les APIs quand elles existent (limites, quotas, authent).



Bibliothèques & outils pour travailler avec les APIs

Consommer une API

- Requests — client HTTP synchrone, JSON, headers, paramètres
- HTTPX — synchrone & asynchrone, support HTTP/2
- Aiohttp — client asynchrone pour forte concurrence réseau
- urllib.request — module standard Python (sans dépendance)

Petit extrait (Requests) :

```
import requests
r = requests.get('https://api.example.com/data', headers={'Authorization': 'Bearer TOKEN'})
data = r.json()
```

Créer / documenter une API

- FastAPI — moderne, ASGI, documentation automatique OpenAPI
- Flask — micro-framework flexible pour APIs simples
- Django REST Framework — APIs complètes au sein de Django
- Outils : Postman / Insomnia / cURL pour tests ; Swagger/OpenAPI pour doc

📌 Notes orales : Montrer l'extrait en direct, expliquer la gestion d'erreurs (status codes) et les limites (rate limits, authent).

Comparaison synthétique & conclusion

Critère	Web Scraping	Web Crawling	API
Objectif	Extraire des données ciblées	Explorer / découvrir des pages	Obtenir des données via un service
Source	Pages HTML	Pages + liens	Service / endpoints
Dépendance au HTML	Oui (souvent)	Souvent	Non (format structuré)
Format résultat	HTML → données extraites	Liste de pages / URLs	JSON / XML / structuré
Navigation	Ciblée	Automatique (links)	Requêtes directes, pas de navigation HTML

Résumé visuel



CRAWLING

« Je découvre les pages » — exploration et découverte.



SCRAPING

« J'extrais les données » — parsing ciblé du contenu.



API

« Je demande les données au service » — format structuré et contractuel.

Conclusion : Ces trois approches sont complémentaires. Préférer les **APIs** quand elles existent, utiliser le **scraping** pour récupérer des champs ciblés, et le **crawling** pour explorer à large échelle — toujours en respectant l'éthique, les CGU et le RGPD.

Sources : Documentation officielle — Python, Requests, BeautifulSoup, Scrapy, Playwright, Selenium, FastAPI, OpenAPI.



Notes orales : Terminer par recommandations pratiques — commencer par vérifier l'API, documenter le pipeline, mettre en place monitoring & tests, et prioriser la conformité légale.